

Argus Security Analysis Report

Repository: https://github.com/arnav-sutraway/Vuln_test

Total Findings: 21

1. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: auth.py, line 22

Evidence: `f"WHERE username = '{username}' "`

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

2. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: auth.py, line 39

Evidence: + "" WHERE email = ""

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

3. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: database.py, line 11

Evidence: query = "SELECT * FROM users WHERE username = " + username + ""

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

4. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: database.py, line 24

Evidence: query = f"SELECT * FROM users WHERE id = {user_id}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

5. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: database.py, line 39

Evidence: `f"WHERE username = '{username}' "`

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

6. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: database.py, line 74

Evidence: query = f"SELECT * FROM products ORDER BY {sort_column}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

7. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: High

Confidence: 88.0%

Location: admin.py, line 55

Evidence: query = f"SELECT * FROM audit_logs WHERE action LIKE '%{search_term}%'"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

8. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: services/users.py, line 10

Evidence: "WHERE email = " + email + " "

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

9. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: services/users.py, line 24

Evidence: query = f"DELETE FROM users WHERE id = {user_id}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

10. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: services/reports.py, line 10

Evidence: `f"WHERE type = '{report_type}' "`

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

11. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: services/reports.py, line 24

Evidence: query = f"SELECT * FROM {table_name}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

12. Stored Cross-Site Scripting in Admin Comments View

Vulnerability Type: Cross-Site Scripting

Severity: High

Confidence: 85.0%

Location: admin.py, line 42

Evidence: `return render_template("admin.html", comments=comments)`

Description: User-supplied comments stored in the database are fetched and passed to the admin template rendering context without ensuring contextual escaping, allowing stored HTML/JavaScript payloads to execute in administrative sessions.

Recommendation: Ensure contextual HTML escaping is applied to comment and username outputs in admin.html, avoiding the use of the safe filter on untrusted stored data.

13. Reflected Cross-Site Scripting via Raw HTML Response

Vulnerability Type: Cross-Site Scripting

Severity: High

Confidence: 95.0%

Location: app.py, line 14

Evidence: `html = f"""\n \n \n Search Results\n You searched for: {query}\n \n \n """`

Description: The search endpoint retrieves the query parameter 'q' from the URL request args and directly formats it into an HTML document string using an f-string before returning it in a text/html Response without sanitization or HTML escaping.

Recommendation: Escape untrusted input using `markupsafe.escape()` or render the search results through Jinja2 templates (`render_template`) with auto-escaping enabled.

14. Reflected Cross-Site Scripting via Markup Safe Bypass

Vulnerability Type: Cross-Site Scripting

Severity: High

Confidence: 95.0%

Location: app.py, line 31

Evidence: `unsafe_content = Markup(f"Welcome, {username}")`

Description: The profile endpoint reads the 'username' query parameter and interpolates it into an HTML string marked explicitly as safe using `markupsafe.Markup`, bypassing automatic escaping mechanisms and allowing arbitrary script execution.

Recommendation: Sanitize or HTML-escape the username parameter using `markupsafe.escape()` before inserting it into markup, or render it via template variables where auto-escaping is active.

15. SQL Injection in User Authentication Query

Vulnerability Type: SQL Injection

Severity: Critical

Confidence: 90.0%

Location: auth.py, line 20

Evidence: query = ("SELECT id, username, role FROM users " f"WHERE username = '{username}' " f"AND password_hash = '{password}'")

Description: User credentials are directly interpolated into the SQL query string using f-strings in the login function, allowing authentication bypass and arbitrary database queries.

Recommendation: Use parameterized queries with placeholders instead of string formatting, for example: conn.execute('SELECT id, username, role FROM users WHERE username = ? AND password_hash = ?', (username, password)).

16. SQL Injection in Password Reset Query

Vulnerability Type: SQL Injection

Severity: High

Confidence: 90.0%

Location: auth.py, line 36

Evidence: query = ("UPDATE users SET password = " + new_password + " WHERE email = " + email + " ")

Description: The reset_password function builds an UPDATE statement via string concatenation with the new_password and email parameters, allowing attackers to manipulate the query structure.

Recommendation: Use parameterized queries with '?' placeholders to bind the new password and email values securely.

17. SQL Injection in authenticate_user

Vulnerability Type: SQL Injection

Severity: Critical

Confidence: 90.0%

Location: database.py, line 37

Evidence: query = (f"SELECT * FROM users " f"WHERE username = '{username}' " f"AND password = '{password}'")

Description: The authenticate_user function uses f-string interpolation to embed the username and password directly into the SQL statement, exposing the query to authentication bypass attacks.

Recommendation: Use parameterized queries: conn.execute('SELECT * FROM users WHERE username = ? AND password = ?', (username, password)).

18. SQL Injection in Product Search

Vulnerability Type: SQL Injection

Severity: High

Confidence: 85.0%

Location: database.py, line 54

Evidence: query = ("SELECT * FROM products " "WHERE name LIKE '%" + search_term + "%' AND category = '" + category + "'")

Description: The search_products function uses string concatenation to incorporate search_term and category into a LIKE clause and WHERE clause, allowing arbitrary SQL execution.

Recommendation: Use parameterized queries with wildcards passed as parameter values: conn.execute('SELECT * FROM products WHERE name LIKE ? AND category = ?', (f'%{search_term}%', category)).

19. SQL Injection in Comment Storage

Vulnerability Type: SQL Injection

Severity: High

Confidence: 85.0%

Location: admin.py, line 18

Evidence: query = ("INSERT INTO comments (username, comment) VALUES ('" + username + "', '" + comment + "')")

Description: The save_comment function constructs an INSERT statement via string concatenation using dynamic username and comment values.

Recommendation: Execute the query using parameterized inputs: conn.execute('INSERT INTO comments (username, comment) VALUES (?, ?)', (username, comment)).

20. SQL Injection in User Search by Filter

Vulnerability Type: SQL Injection

Severity: High

Confidence: 85.0%

Location: services/users.py, line 8

Evidence: query = ("SELECT * FROM users " "WHERE email = '" + email + "' " "AND role = '" + role + "'")

Description: The find_users function constructs a dynamic SELECT query by concatenating the email and role arguments directly into the SQL string.

Recommendation: Use parameterized queries: conn.execute('SELECT * FROM users WHERE email = ? AND role = ?', (email, role)).

21. SQL Injection in Report Generation

Vulnerability Type: SQL Injection

Severity: High

Confidence: 85.0%

Location: services/reports.py, line 8

Evidence: query = ("SELECT * FROM reports " f"WHERE type = '{report_type}' " f"AND owner_id = {owner_id}")

Description: The generate_report function uses f-strings to interpolate report_type and owner_id directly into the SQL WHERE clause.

Recommendation: Use parameterized queries: conn.execute('SELECT * FROM reports WHERE type = ? AND owner_id = ?', (report_type, owner_id)).